

computer-system-security
1.0.0

Generated by Doxygen 1.13.2

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	3
2.1 Class List	3
3 Class Documentation	5
3.1 org.example.encryption.AESCrypto Class Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	5
3.1.2.1 decrypt()	5
3.1.2.2 encrypt()	5
3.1.2.3 generatelv()	6
3.1.2.4 getKey()	6
3.1.2.5 restorelv()	6
3.2 org.example.crypto.CryptoMainWindow Class Reference	7
3.2.1 Detailed Description	7
3.2.2 Constructor & Destructor Documentation	7
3.2.2.1 CryptoMainWindow()	7
3.3 org.example.crypto.KeyGeneratorBody Class Reference	7
3.3.1 Detailed Description	7
3.3.2 Constructor & Destructor Documentation	7
3.3.2.1 KeyGeneratorBody()	7
3.3.3 Member Function Documentation	8
3.3.3.1 generatePair()	8
3.4 org.example.crypto.KeyGeneratorWindow Class Reference	8
3.4.1 Detailed Description	9
3.4.2 Constructor & Destructor Documentation	9
3.4.2.1 KeyGeneratorWindow()	9
3.5 org.example.crypto.PDFCryptoBody Class Reference	9
3.5.1 Detailed Description	9
3.5.2 Constructor & Destructor Documentation	9
3.5.2.1 PDFCryptoBody()	9
3.5.3 Member Function Documentation	10
3.5.3.1 GetPrivateKey()	10
3.5.3.2 ScanForKey()	10
3.5.3.3 SignPDF()	10
3.6 org.example.crypto.PDFCryptoWindow Class Reference	11
3.6.1 Detailed Description	11
3.6.2 Constructor & Destructor Documentation	11
3.6.2.1 PDFCryptoWindow()	11
3.7 org.example.verify.PDFVerifyBody Class Reference	12
3.7.1 Detailed Description	12

3.7.2 Member Function Documentation	12
3.7.2.1 getPublicKey()	12
3.7.2.2 verifyPDF()	12
3.8 org.example.verify.PDFVerifyWindow Class Reference	13
3.8.1 Detailed Description	13
3.8.2 Constructor & Destructor Documentation	13
3.8.2.1 PDFVerifyWindow()	13
3.9 org.example.encryption.RSACrypto Class Reference	14
3.9.1 Detailed Description	14
3.9.2 Member Function Documentation	14
3.9.2.1 getPair()	14
3.10 org.example.SignAppMain Class Reference	14
3.10.1 Detailed Description	14
3.10.2 Member Function Documentation	14
3.10.2.1 main()	14
3.11 org.example.encryption.SignatureManager Class Reference	15
3.11.1 Detailed Description	15
3.11.2 Member Function Documentation	15
3.11.2.1 changeOneByte()	15
3.11.2.2 getChosenHash()	16
3.11.2.3 sign()	16
3.11.2.4 signPDF()	16
3.11.2.5 verifyPDF()	17
3.12 org.example.usbseeker.USBSeeker Class Reference	17
3.12.1 Detailed Description	18
3.12.2 Member Function Documentation	18
3.12.2.1 findPendives()	18
3.12.2.2 getKeyPath()	18
3.13 org.example.VerificationAppMain Class Reference	18
3.13.1 Detailed Description	19
3.13.2 Member Function Documentation	19
3.13.2.1 main()	19

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

org.example.encryption.AESCrypto	5
JFrame	
org.example.crypto.CryptoMainWindow	7
org.example.verify.PDFVerifyWindow	13
JPanel	
org.example.crypto.KeyGeneratorWindow	8
org.example.crypto.PDFCryptoWindow	11
org.example.crypto.KeyGeneratorBody	7
org.example.crypto.PDFCryptoBody	9
org.example.verify.PDFVerifyBody	12
org.example.encryption.RSACrypto	14
org.example.SignAppMain	14
SignatureInterface	
org.example.encryption.SignatureManager	15
org.example.usbseeker.USBSeeker	17
org.example.VerificationAppMain	18

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

org.example.encryption.AESCrypto	5
org.example.crypto.CryptoMainWindow	7
org.example.crypto.KeyGeneratorBody	7
org.example.crypto.KeyGeneratorWindow	8
org.example.crypto.PDFCryptoBody	9
org.example.crypto.PDFCryptoWindow	11
org.example.verify.PDFVerifyBody	12
org.example.verify.PDFVerifyWindow	13
org.example.encryption.RSACrypto	14
org.example.SignAppMain	14
org.example.encryption.SignatureManager	15
org.example.usbseeker.USBSeeker	17
org.example.VerificationAppMain	18

Chapter 3

Class Documentation

3.1 org.example.encryption.AESCrypto Class Reference

Public Member Functions

- **AESCrypto** (String type)
- SecretKey **getKey** (String pin, Integer iterations, Integer keyLength, String charSet) throws Exception
- IvParameterSpec **generateIv** ()
- void **restoreIv** (byte[] iv)
- byte[] **encrypt** (byte[] text, SecretKey key) throws Exception
- byte[] **decrypt** (byte[] complexText, SecretKey key) throws Exception

3.1.1 Detailed Description

AES Encryption/Decryption Utility with Secure IV Handling.

3.1.2 Member Function Documentation

3.1.2.1 decrypt()

```
byte[] org.example.encryption.AESCrypto.decrypt (  
    byte[] complexText,  
    SecretKey key) throws Exception
```

Decrypts AES-encrypted data.

Parameters

<i>complexText</i>	The IV + Encrypted content.
<i>key</i>	The AES secret key.

Returns

Decrypted plaintext data.

3.1.2.2 encrypt()

```
byte[] org.example.encryption.AESCrypto.encrypt (  
    byte[] text,  
    SecretKey key) throws Exception
```

Encrypts data using AES with a unique IV.

Parameters

<i>text</i>	The plaintext data.
<i>key</i>	The AES secret key.

Returns

IV + Encrypted data.

3.1.2.3 generateIv()

```
IvParameterSpec org.example.encryption.AESCrypto.generateIv ()
```

Generates a random IV.

Returns

New IV as IvParameterSpec.

3.1.2.4 getKey()

```
SecretKey org.example.encryption.AESCrypto.getKey (  
    String pin,  
    Integer iterations,  
    Integer keyLength,  
    String charSet) throws Exception
```

Generates a SecretKey using PBKDF2 with HMAC-SHA256.

Parameters

<i>pin</i>	The password/PIN used for key derivation.
<i>iterations</i>	Number of PBKDF2 iterations.
<i>keyLength</i>	Length of the key in bits (e.g., 256 for AES-256).

Returns

Derived SecretKey.

3.1.2.5 restoreIv()

```
void org.example.encryption.AESCrypto.restoreIv (  
    byte[] iv)
```

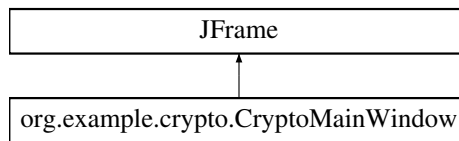
Restores a IV

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/encryption/AESCrypto.java

3.2 org.example.crypto.CryptoMainWindow Class Reference

Inheritance diagram for org.example.crypto.CryptoMainWindow:



Public Member Functions

- [CryptoMainWindow](#) ()

3.2.1 Detailed Description

[CryptoMainWindow](#) serves as the main graphical user interface (GUI) for the RSA Key Generator and PDF Signing application. It provides navigation between different features using a CardLayout.

3.2.2 Constructor & Destructor Documentation

3.2.2.1 CryptoMainWindow()

```
org.example.crypto.CryptoMainWindow.CryptoMainWindow ()
```

Constructor to initialize the [CryptoMainWindow](#). Sets up the UI components and event listeners.

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/crypto/CryptoMainWindow.↔
java

3.3 org.example.crypto.KeyGeneratorBody Class Reference

Public Member Functions

- [KeyGeneratorBody](#) (Integer keyLength)
- void [generatePair](#) (String pin, String privatePath, String publicPath, String keyFileName) throws Exception

3.3.1 Detailed Description

[KeyGeneratorBody](#) is responsible for generating RSA key pairs and encrypting private keys. It supports AES encryption for securing private keys and provides file-writing functionality.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 KeyGeneratorBody()

```
org.example.crypto.KeyGeneratorBody.KeyGeneratorBody (  
    Integer keyLength)
```

Constructor to initialize the key generator with a specified key length.

Parameters

<i>keyLength</i>	Length of the RSA key pair (e.g., 2048, 4096 bits).
------------------	---

3.3.3 Member Function Documentation

3.3.3.1 generatePair()

```
void org.example.crypto.KeyGeneratorBody.generatePair (
    String pin,
    String privatePath,
    String publicPath,
    String keyFileName) throws Exception
```

Generates an RSA key pair, encrypts the private key, and writes both keys to specified locations.

Parameters

<i>pin</i>	User-provided PIN used for AES encryption of the private key.
<i>privatePath</i>	Directory where the encrypted private key should be saved.
<i>publicPath</i>	Directory where the public key should be saved.
<i>keyFileName</i>	Name of the encrypted private key file.

Exceptions

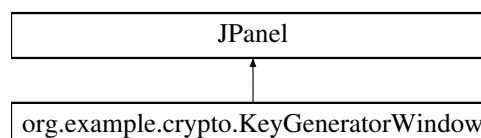
<i>Exception</i>	If an error occurs during key generation or encryption.
------------------	---

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/crypto/KeyGeneratorBody.↔
java

3.4 org.example.crypto.KeyGeneratorWindow Class Reference

Inheritance diagram for org.example.crypto.KeyGeneratorWindow:



Public Member Functions

- [KeyGeneratorWindow](#) (CardLayout cardLayout, JPanel cardPanel, String keyFileName)

3.4.1 Detailed Description

[KeyGeneratorWindow](#) is a GUI panel for generating RSA key pairs. It allows users to choose storage locations for keys and input a PIN for encryption.

3.4.2 Constructor & Destructor Documentation

3.4.2.1 KeyGeneratorWindow()

```
org.example.crypto.KeyGeneratorWindow.KeyGeneratorWindow (  
    CardLayout cardLayout,  
    JPanel cardPanel,  
    String keyFileName)
```

Constructor to initialize the key generator window.

Parameters

<i>cardLayout</i>	Parent CardLayout for navigation.
<i>cardPanel</i>	Parent JPanel containing this window.
<i>keyFileName</i>	Name of the private key file.

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/crypto/KeyGeneratorWindow.java

3.5 org.example.crypto.PDFCryptoBody Class Reference

Public Member Functions

- [PDFCryptoBody](#) (Integer keyLength, String keyFileName)
- PrivateKey [GetPrivateKey](#) (String pin, String keyPath) throws Exception
- String [ScanForKey](#) (String keyFileName)
- boolean [SignPDF](#) (String pin, String origPath, String signedPath, String keyPath) throws Exception

3.5.1 Detailed Description

[PDFCryptoBody](#) handles the cryptographic operations for signing PDFs. It retrieves the private RSA key (secured with AES) and signs a PDF document.

3.5.2 Constructor & Destructor Documentation

3.5.2.1 PDFCryptoBody()

```
org.example.crypto.PDFCryptoBody.PDFCryptoBody (  
    Integer keyLength,  
    String keyFileName)
```

Constructor initializes the cryptographic body with a specified key length.

Parameters

<i>keyLength</i>	Length of the RSA key (default: 4096 bits if null).
<i>keyFileName</i>	Name of the private key file (unused but included for future extensibility).

3.5.3 Member Function Documentation

3.5.3.1 GetPrivateKey()

```
PrivateKey org.example.crypto.PDFCryptoBody.GetPrivateKey (  
    String pin,  
    String keyPath) throws Exception
```

Retrieves and decrypts the private RSA key from a given key file path.

Parameters

<i>pin</i>	The user-provided PIN for decryption.
<i>keyPath</i>	The file path of the encrypted private key.

Returns

The decrypted RSA private key.

Exceptions

<i>Exception</i>	If the key cannot be found or the PIN is incorrect.
------------------	---

3.5.3.2 ScanForKey()

```
String org.example.crypto.PDFCryptoBody.ScanForKey (  
    String keyFileName)
```

Scans connected USB drives for the private key file.

Parameters

<i>keyFileName</i>	The name of the private key file to search for.
--------------------	---

Returns

The file path of the key if found, otherwise null.

3.5.3.3 SignPDF()

```
boolean org.example.crypto.PDFCryptoBody.SignPDF (  
    String pin,  
    String origPath,  
    String signedPath,  
    String keyPath) throws Exception
```

Signs a PDF file using the retrieved RSA private key.

Parameters

<i>pin</i>	The user-provided PIN for key decryption.
<i>origPath</i>	The path to the original PDF file.
<i>signedPath</i>	The destination path for the signed PDF.
<i>keyPath</i>	The file path of the encrypted private key.

Returns

True if signing is successful, otherwise false.

Exceptions

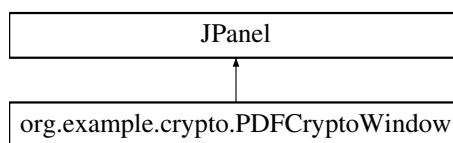
<i>Exception</i>	If an error occurs during signing.
------------------	------------------------------------

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/crypto/PDFCryptoBody.java

3.6 org.example.crypto.PDFCryptoWindow Class Reference

Inheritance diagram for org.example.crypto.PDFCryptoWindow:



Public Member Functions

- [PDFCryptoWindow](#) (CardLayout cardLayout, JPanel cardPanel, String keyFileName)

3.6.1 Detailed Description

[PDFCryptoWindow](#) provides a graphical interface for signing PDF files. Users can select an original file, specify the signed file location, enter a PIN, and scan for a private key to sign the document.

3.6.2 Constructor & Destructor Documentation

3.6.2.1 PDFCryptoWindow()

```
org.example.crypto.PDFCryptoWindow.PDFCryptoWindow (
    CardLayout cardLayout,
    JPanel cardPanel,
    String keyFileName)
```

Constructor to initialize the UI for PDF signing.

Parameters

<i>cardLayout</i>	Parent CardLayout for navigation.
<i>cardPanel</i>	Parent JPanel container.
<i>keyFileName</i>	Name of the private key file to search for.

The documentation for this class was generated from the following file:

- `/Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/crypto/PDFCryptoWindow.java`↔

3.7 org.example.verify.PDFVerifyBody Class Reference

Public Member Functions

- `PublicKey` [getPublicKey](#) (`FileInputStream` *keyStream*) throws `Exception`
- `boolean` [verifyPDF](#) (`File` *file*) throws `Exception`

3.7.1 Detailed Description

The [PDFVerifyBody](#) class is responsible for verifying the authenticity of PDF documents by validating their digital signatures using a public RSA key. This class utilizes a [SignatureManager](#) instance to handle the signature verification process.

3.7.2 Member Function Documentation

3.7.2.1 getPublicKey()

```
PublicKey org.example.verify.PDFVerifyBody.getPublicKey (  
    FileInputStream keyStream) throws Exception
```

Retrieves a public RSA key from a given `FileInputStream`. This method reads the public key bytes and generates the corresponding `PublicKey` object.

Parameters

<i>keyStream</i>	The <code>FileInputStream</code> containing the public key.
------------------	---

Returns

The `PublicKey` generated from the input stream.

Exceptions

<i>Exception</i>	If any error occurs while processing the key.
------------------	---

3.7.2.2 verifyPDF()

```
boolean org.example.verify.PDFVerifyBody.verifyPDF (  
    File file) throws Exception
```

Verifies the authenticity of a PDF document by checking its digital signature. The method retrieves the public key, verifies the PDF using the [SignatureManager](#), and prints a message indicating whether the verification was successful.

Parameters

<i>file</i>	The PDF file to verify.
-------------	-------------------------

Returns

True if the signature verification is successful, false otherwise.

Exceptions

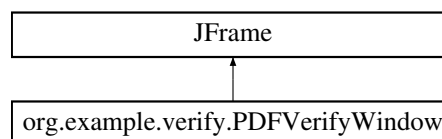
<i>Exception</i>	If an error occurs during verification (e.g., file not found, invalid key).
------------------	---

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/verify/PDFVerifyBody.java

3.8 org.example.verify.PDFVerifyWindow Class Reference

Inheritance diagram for org.example.verify.PDFVerifyWindow:

**Public Member Functions**

- [PDFVerifyWindow](#) ()

3.8.1 Detailed Description

The [PDFVerifyWindow](#) class provides a graphical user interface (GUI) for verifying the authenticity of PDF files using digital signatures. It allows users to select a PDF file from their filesystem and verify its integrity and signature validity. The UI comprises components for selecting a file, displaying results, and showing helpful tooltips.

3.8.2 Constructor & Destructor Documentation

3.8.2.1 PDFVerifyWindow()

```
org.example.verify.PDFVerifyWindow.PDFVerifyWindow ()
```

Constructor initializes the PDF verification window, sets up the GUI components, and configures actions such as choosing a PDF and verifying the document.

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/verify/PDFVerifyWindow.java

3.9 org.example.encryption.RSACrypto Class Reference

Public Member Functions

- `KeyPair` `getPair` (Integer *length*) throws `Exception`

3.9.1 Detailed Description

RSA Encryption/Decryption Utility. Used to generate RSA key pairs.

3.9.2 Member Function Documentation

3.9.2.1 `getPair()`

```
KeyPair org.example.encryption.RSACrypto.getPair (  
    Integer length) throws Exception
```

Generates an RSA key pair of the specified length.

Parameters

<i>length</i>	The key length (e.g., 2048, 3072, 4096).
---------------	--

Returns

The generated RSA key pair.

The documentation for this class was generated from the following file:

- `/Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/encryption/RSACrypto.java`

3.10 org.example.SignAppMain Class Reference

Static Public Member Functions

- static void `main` (String[] *args*)

3.10.1 Detailed Description

Main class for the Sign PDF application.

3.10.2 Member Function Documentation

3.10.2.1 `main()`

```
static void org.example.SignAppMain.main (  
    String[] args) [static]
```

Main method to start the application.

Parameters

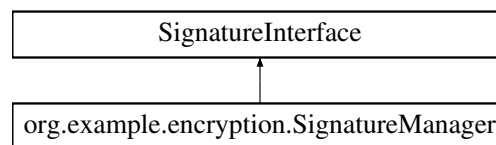
<i>args</i>	Command-line arguments (not used in this application).
-------------	--

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/SignAppMain.java

3.11 org.example.encryption.SignatureManager Class Reference

Inheritance diagram for org.example.encryption.SignatureManager:



Public Member Functions

- byte[] [getChosenHash](#) (PDDocument document) throws Exception
- boolean [signPDF](#) (String origPath, String signedPath, PrivateKey privateRSA) throws Exception
- byte[] [sign](#) (InputStream content) throws IOException
- boolean [verifyPDF](#) (File filePDF, PublicKey publicRSA) throws Exception
- void [changeOneByte](#) (File filePDF) throws Exception

3.11.1 Detailed Description

The [SignatureManager](#) class provides functionality for signing, verifying, and manipulating PDF documents. It implements the SignatureInterface and uses RSA cryptography for signing operations.

3.11.2 Member Function Documentation

3.11.2.1 changeOneByte()

```
void org.example.encryption.SignatureManager.changeOneByte (
    File filePDF) throws Exception
```

Modifies one byte of the document's content to simulate corruption.

Parameters

<i>filePDF</i>	The PDF file to modify.
----------------	-------------------------

Exceptions

<i>Exception</i>	If an error occurs during the modification process.
------------------	---

3.11.2.2 getChosenHash()

```
byte[] org.example.encryption.SignatureManager.getChosenHash (  
    PDDocument document) throws Exception
```

Hashes the content of the PDF document using SHA-256.

Parameters

<i>document</i>	The PDF document to hash.
-----------------	---------------------------

Returns

The SHA-256 hash of the document content.

Exceptions

<i>Exception</i>	If there is an error during the hashing process.
------------------	--

3.11.2.3 sign()

```
byte[] org.example.encryption.SignatureManager.sign (  
    InputStream content) throws IOException
```

Signs the hashed content of a document with the private key.

Parameters

<i>content</i>	The content to sign.
----------------	----------------------

Returns

The signed content (digital signature).

Exceptions

<i>IOException</i>	If an I/O error occurs during signing.
--------------------	--

3.11.2.4 signPDF()

```
boolean org.example.encryption.SignatureManager.signPDF (  
    String origPath,  
    String signedPath,  
    PrivateKey privateKey) throws Exception
```

Signs the PDF document and saves the signed document to a new path.

Parameters

<i>origPath</i>	The path to the original PDF.
<i>signedPath</i>	The path to save the signed PDF.
<i>privateRSA</i>	The private key used to sign the document.

Returns

True if signing was successful, false otherwise.

Exceptions

<i>Exception</i>	If there is an error during the signing process.
------------------	--

3.11.2.5 verifyPDF()

```
boolean org.example.encryption.SignatureManager.verifyPDF (
    File filePDF,
    PublicKey publicRSA) throws Exception
```

Verifies the signature of a signed PDF file.

Parameters

<i>filePDF</i>	The PDF file to verify.
<i>publicRSA</i>	The public key used to verify the signature.

Returns

True if the signature is valid, false otherwise.

Exceptions

<i>Exception</i>	If there is an error during the verification process.
------------------	---

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/encryption/SignatureManager.java

3.12 org.example.usbseeker.USBSeeker Class Reference

Public Member Functions

- List< File > [findPendives](#) ()
- String [getKeyPath](#) (String keyFileName)

3.12.1 Detailed Description

The [USBSeeker](#) class is responsible for detecting connected USB drives and searching for specific files within those drives. It is designed to operate across multiple operating systems including Windows, macOS, and Linux.

This class provides functionality to:

- Identify USB drives based on operating system-specific criteria.
- Search connected USB drives for a file with a specific name.
- Handle differences in file system structure across platforms.

3.12.2 Member Function Documentation

3.12.2.1 findPendrives()

```
List< File > org.example.usbseeker.USBSeeker.findPendrives ()
```

Finds all connected USB drives (pendrives) based on the operating system.

Returns

A list of File objects representing the pendrives found on the system.

3.12.2.2 getKeyPath()

```
String org.example.usbseeker.USBSeeker.getKeyPath (  
    String keyFileName)
```

Searches for the key file on all detected USB drives.

Parameters

<i>keyFileName</i>	The name of the key file to search for.
--------------------	---

Returns

The full path of the key file if found, or "None" if not found.

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/usbseeker/USBSeeker.java

3.13 org.example.VerificationAppMain Class Reference

Static Public Member Functions

- static void [main](#) (String[] args)

3.13.1 Detailed Description

Main class for the Signature Verification application.

3.13.2 Member Function Documentation

3.13.2.1 main()

```
static void org.example.VerificationAppMain.main (  
    String[] args)    [static]
```

Main method to launch the Signature Verification application.

Parameters

<i>args</i>	Command-line arguments (not used in this application).
-------------	--

The documentation for this class was generated from the following file:

- /Users/nickpozniak/PG/Computer-System-Security/src/main/java/org/example/VerificationAppMain.java

Index

- changeOneByte
 - org.example.encryption.SignatureManager, 15
- CryptoMainWindow
 - org.example.crypto.CryptoMainWindow, 7
- decrypt
 - org.example.encryption.AESCrypto, 5
- encrypt
 - org.example.encryption.AESCrypto, 5
- findPendives
 - org.example.usbseeker.USBSeeker, 18
- generatelv
 - org.example.encryption.AESCrypto, 6
- generatePair
 - org.example.crypto.KeyGeneratorBody, 8
- getChosenHash
 - org.example.encryption.SignatureManager, 16
- getKey
 - org.example.encryption.AESCrypto, 6
- getKeyPath
 - org.example.usbseeker.USBSeeker, 18
- getPair
 - org.example.encryption.RSACrypto, 14
- GetPrivateKey
 - org.example.crypto.PDFCryptoBody, 10
- getPublicKey
 - org.example.verify.PDFVerifyBody, 12
- KeyGeneratorBody
 - org.example.crypto.KeyGeneratorBody, 7
- KeyGeneratorWindow
 - org.example.crypto.KeyGeneratorWindow, 9
- main
 - org.example.SignAppMain, 14
 - org.example.VerificationAppMain, 19
- org.example.crypto.CryptoMainWindow, 7
 - CryptoMainWindow, 7
- org.example.crypto.KeyGeneratorBody, 7
 - generatePair, 8
 - KeyGeneratorBody, 7
- org.example.crypto.KeyGeneratorWindow, 8
 - KeyGeneratorWindow, 9
- org.example.crypto.PDFCryptoBody, 9
 - GetPrivateKey, 10
 - PDFCryptoBody, 9
 - ScanForKey, 10
- SignPDF, 10
- org.example.crypto.PDFCryptoWindow, 11
 - PDFCryptoWindow, 11
- org.example.encryption.AESCrypto, 5
 - decrypt, 5
 - encrypt, 5
 - generatelv, 6
 - getKey, 6
 - restorelv, 6
- org.example.encryption.RSACrypto, 14
 - getPair, 14
- org.example.encryption.SignatureManager, 15
 - changeOneByte, 15
 - getChosenHash, 16
 - sign, 16
 - signPDF, 16
 - verifyPDF, 17
- org.example.SignAppMain, 14
 - main, 14
- org.example.usbseeker.USBSeeker, 17
 - findPendives, 18
 - getKeyPath, 18
- org.example.VerificationAppMain, 18
 - main, 19
- org.example.verify.PDFVerifyBody, 12
 - getPublicKey, 12
 - verifyPDF, 12
- org.example.verify.PDFVerifyWindow, 13
 - PDFVerifyWindow, 13
- PDFCryptoBody
 - org.example.crypto.PDFCryptoBody, 9
- PDFCryptoWindow
 - org.example.crypto.PDFCryptoWindow, 11
- PDFVerifyWindow
 - org.example.verify.PDFVerifyWindow, 13
- restorelv
 - org.example.encryption.AESCrypto, 6
- ScanForKey
 - org.example.crypto.PDFCryptoBody, 10
- sign
 - org.example.encryption.SignatureManager, 16
- SignPDF
 - org.example.crypto.PDFCryptoBody, 10
- signPDF
 - org.example.encryption.SignatureManager, 16
- verifyPDF

`org.example.encryption.SignatureManager`, [17](#)
`org.example.verify.PDFVerifyBody`, [12](#)